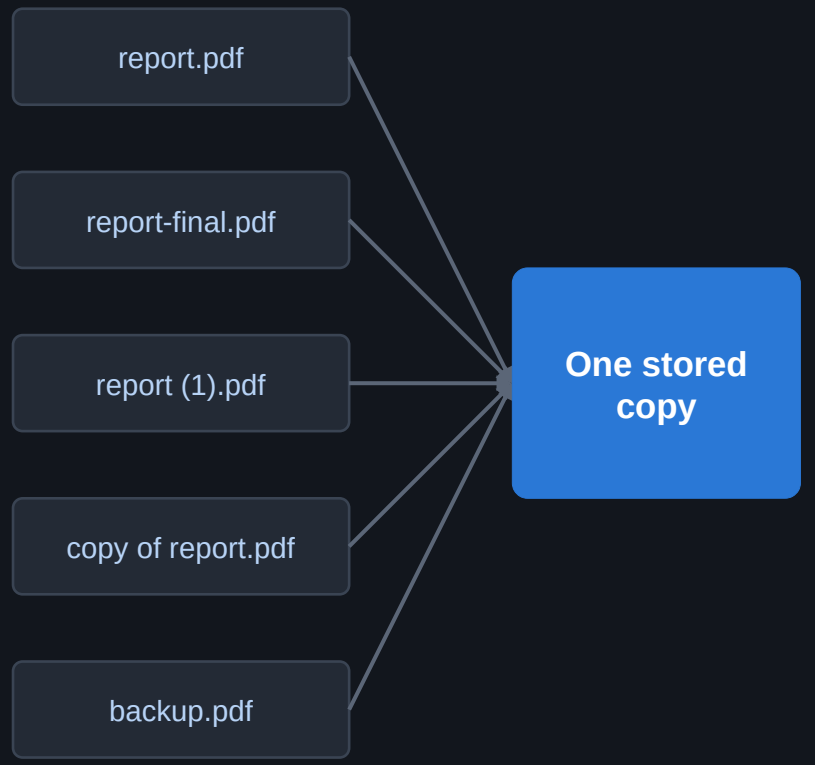
HDFS Optimizer

HDFS-Based Distributed File Replication, Deduplication and Storage Optimization System

Store it once. Replicate it safely. Prove every byte.



[Student name] · [Roll number] · [Department] · [University]
Guide: [Supervisor] · Session: [Academic session]

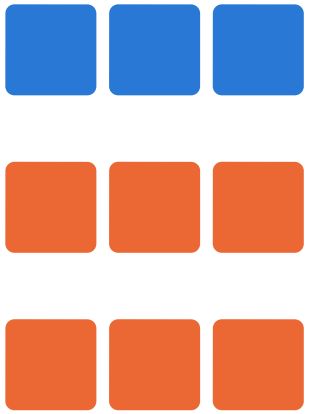
HDFS protects blocks, but it cannot see duplicate files

Plain HDFS: three uploads of one book

report.pdf

report-final.pdf

copy of report.pdf



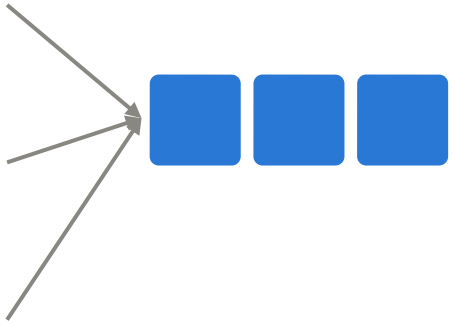
9 block copies stored

With HDFS Optimizer

report.pdf

report-final.pdf

copy of report.pdf



3 block copies, same fault tolerance

Each square is one replica of one block. Blue copies provide safety. Orange copies come only from duplicate uploads.

Every accidental copy is multiplied by the replication factor

240 MiB

consumed by 9 books with 5 extra copies each, stored as plain uploads

40 MiB

for the same logical files, deduplicated, at the same replication factor

3 ×

every stored byte is kept on three DataNodes, including accidental copies

Why this is hard to fix by hand

No visibility

HDFS has no view of identical content across files.

Risky cleanup

Deleting a "duplicate" by hand can remove what someone needs.

Confusing numbers

Logical size, stored size and disk use are three different things.

Measured with authentic files on the test cluster. Duplicates were created for the experiment.

A layer above HDFS that stores each distinct content once

1

Whole-file deduplication

SHA-256 identity. Any number of names, one stored object.

2

Chunk-level deduplication

Fixed-size chunks shared across different files.

3

Safe optimization

Dry run first. Each file is verified before its old copy goes.

4

Reference-aware deletion

Recoverable deletes. Shared content stays while it is needed.

5

Replication management

Real factor changes, confirmed with fsck and block locations.

6

Honest metrics

Every headline value says: measured, derived or estimated.

What the project set out to achieve

Reduce

- Detect exact duplicates with SHA-256
- Find identical chunks across files
- Store unique data in content-addressed form

Protect

- Reconstruct originals exactly
- Verify with cryptographic checksums
- Delete safely, and recover

Operate

- Manage supported replication settings
- Monitor NameNode and DataNode health
- Show storage and savings clearly

Prove

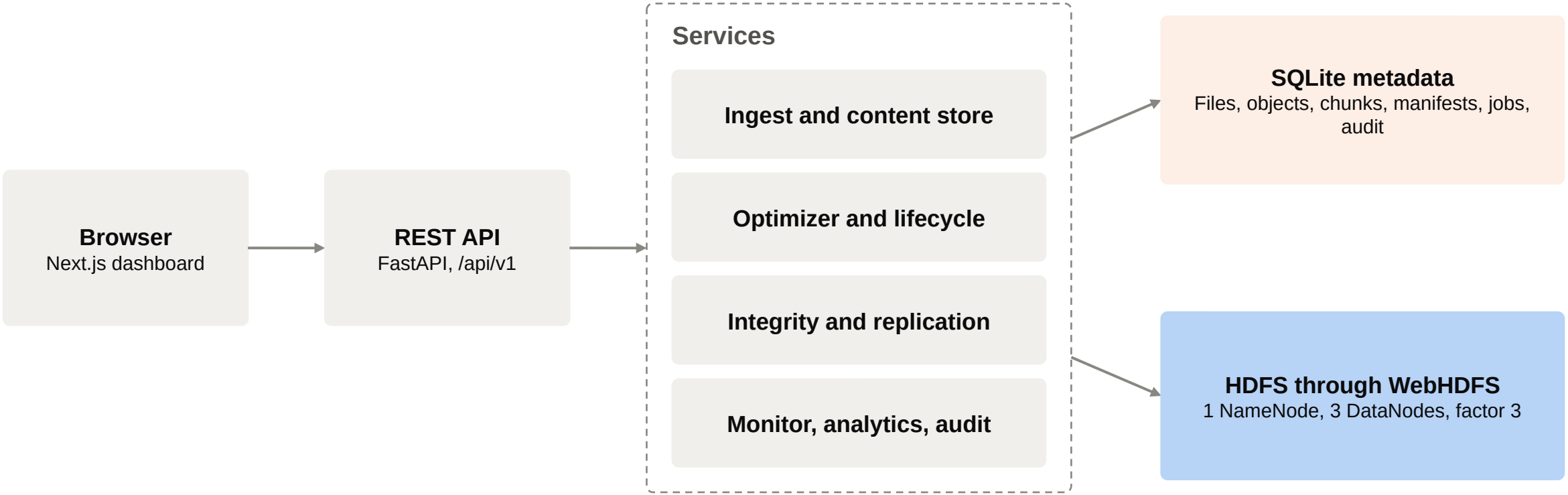
- Evaluate on authentic datasets
- Compare storage visually
- Document design and results

Keep the redundancy that protects data. Remove the redundancy that only costs space.

A deliberately ordinary, inspectable technology stack

Interface	Next.js 16	React 19	TypeScript	Tailwind + shadcn/ui	Recharts
API and logic	Python 3.12	FastAPI	Pydantic	HTTPX	hashlib SHA-256
Metadata	SQLite (WAL)	SQLAlchemy 2	Alembic migrations		
Storage	Apache Hadoop 3.5.0	HDFS	WebHDFS REST	JMX and fsck	
Platform and tests	Docker Compose	Colima	Pytest	Playwright	Typst

Bytes reach HDFS first, and metadata commits afterwards

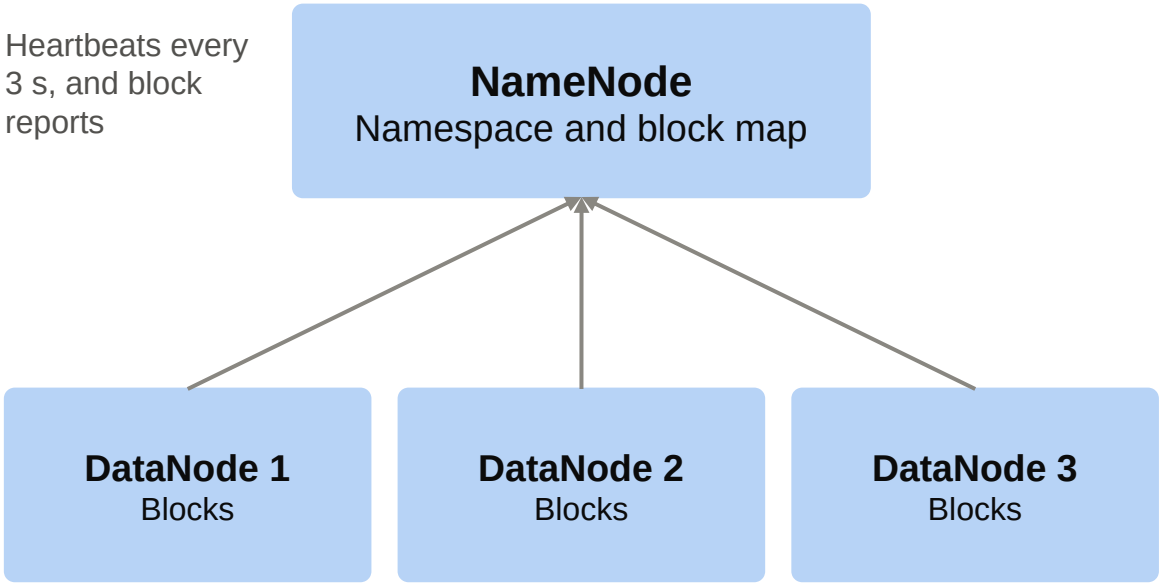


Write order: 1 stage in HDFS 2 check bytes 3 rename atomically 4 commit metadata

A crash can leave an unreferenced file, but never metadata pointing at missing bytes.

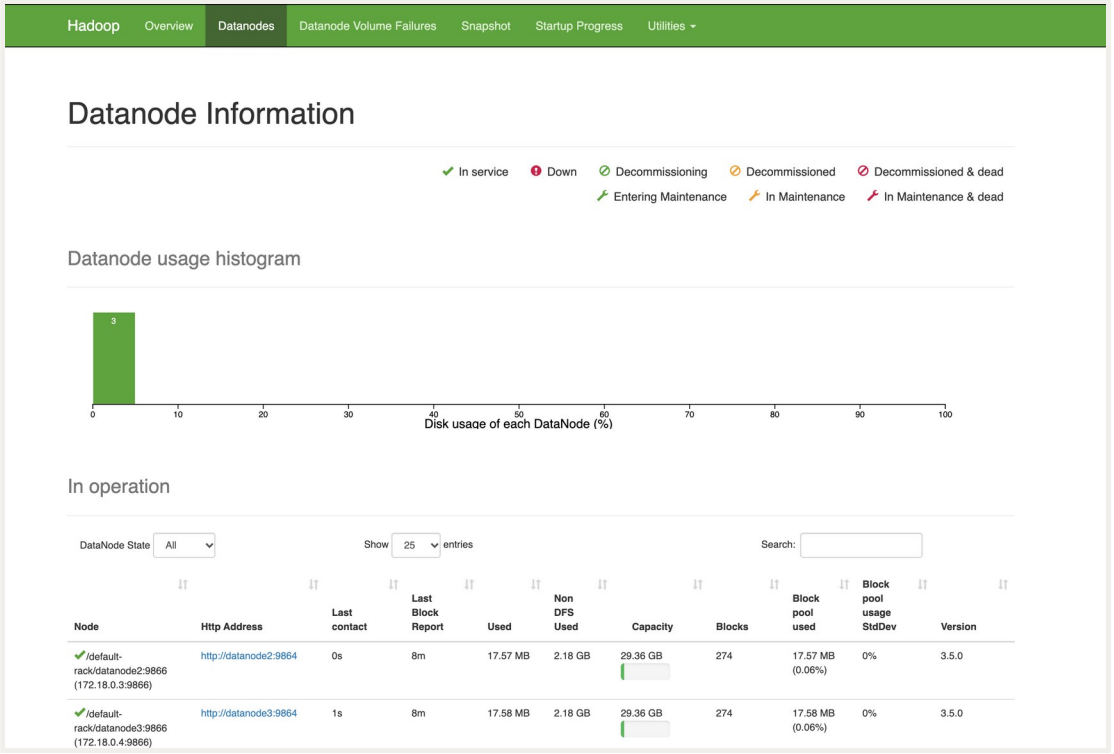
A real HDFS cluster: one NameNode and three DataNodes

Heartbeats every 3 s, and block reports



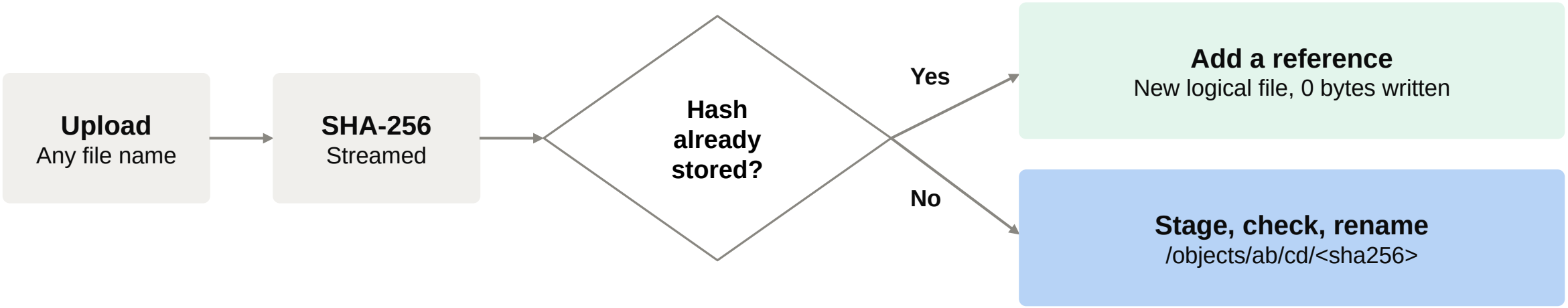
Each block is stored on 3 DataNodes. Stopping a DataNode leaves files readable, which we tested for real.

Limit: all nodes share one laptop and one disk.



Hadoop's own NameNode interface, showing this cluster

Whole-file deduplication: same hash, no new bytes



20 names

of one authentic file, in a single test

1 HDFS file

stored for all of them

20 of 20

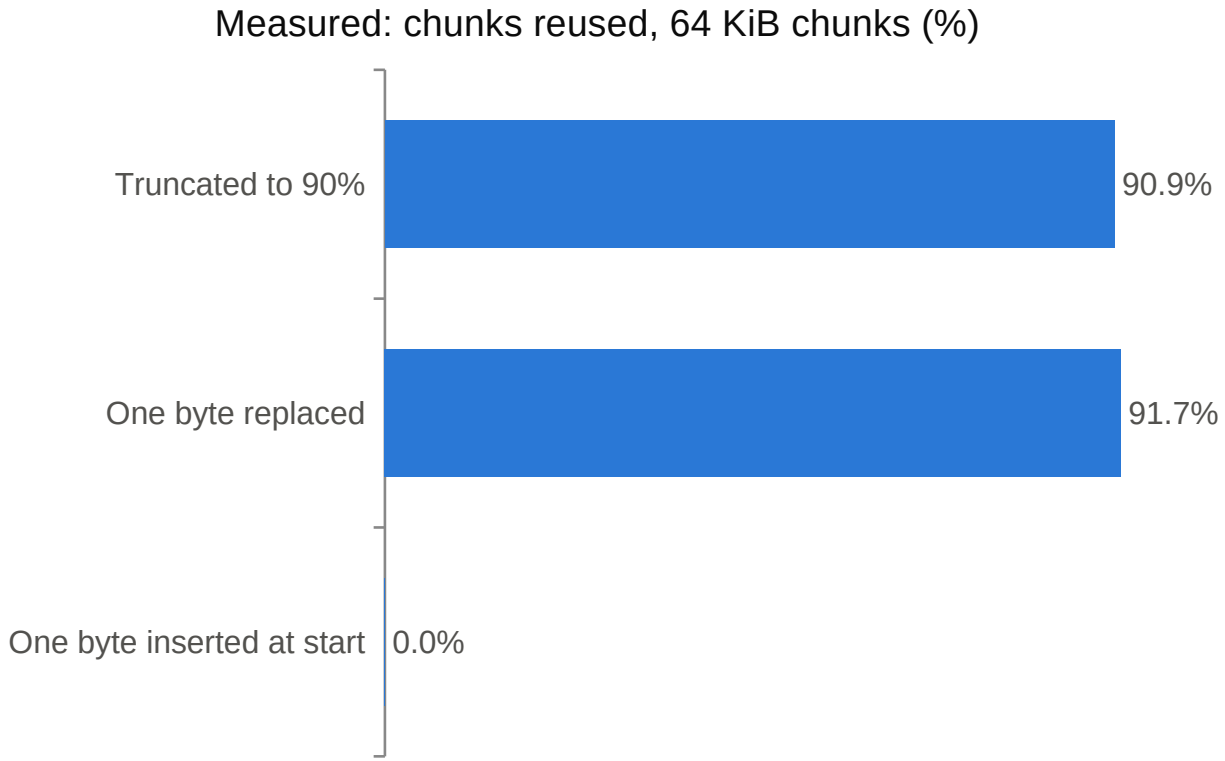
names passed SHA-256 verification

Chunks find overlap between different files



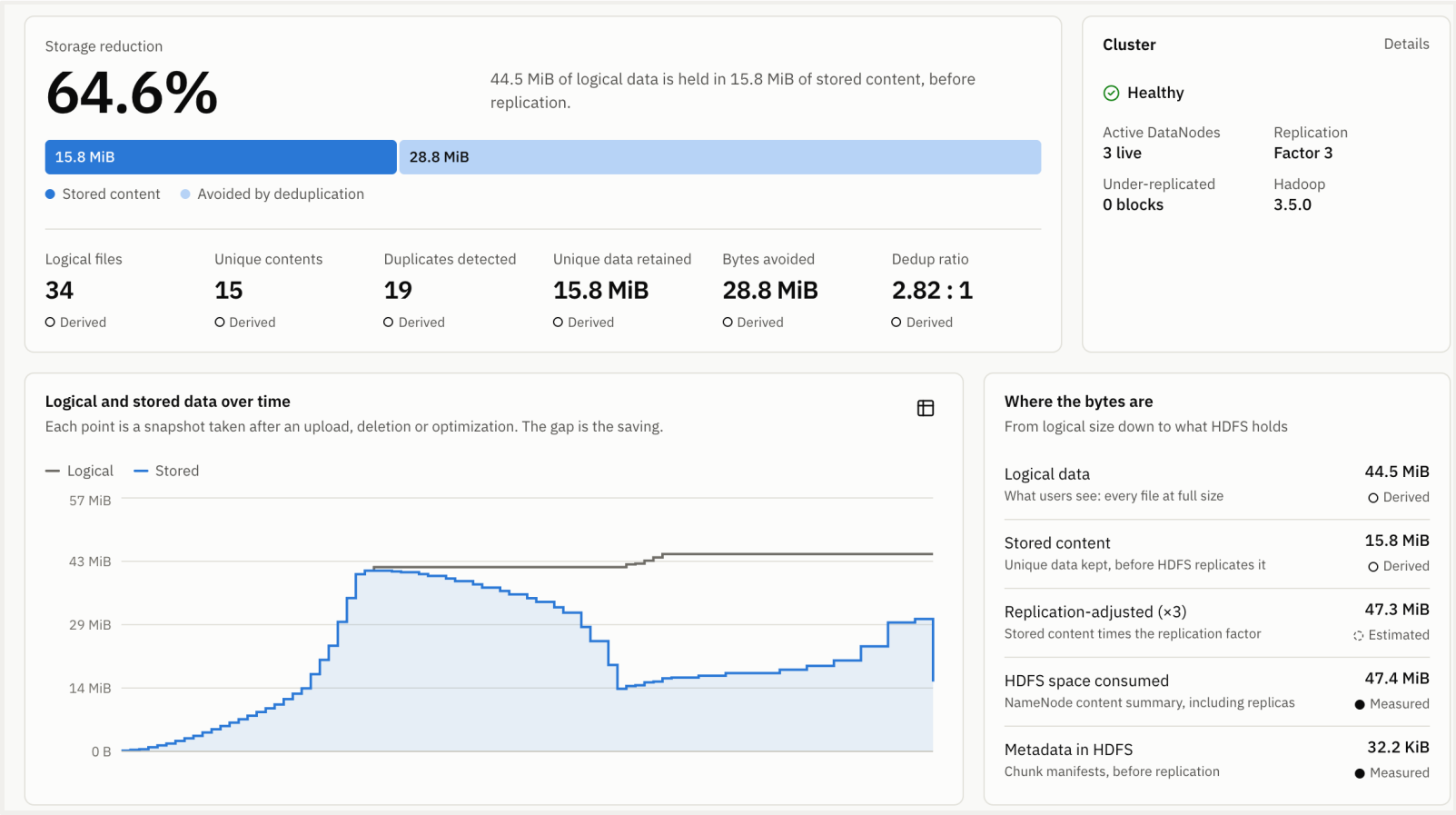
Stored once each: c1, c2, c3, c4, c5. A manifest lists each file's chunks in order, and rebuilds it exactly.

Chunks are application-level HDFS files, not HDFS blocks.



One inserted byte shifts every boundary. Content-defined chunking would fix this; it is future work.

The dashboard says how every number is known



Derived

Exact, from the metadata database. Excludes replication.

Measured

Reported by HDFS at the time shown.

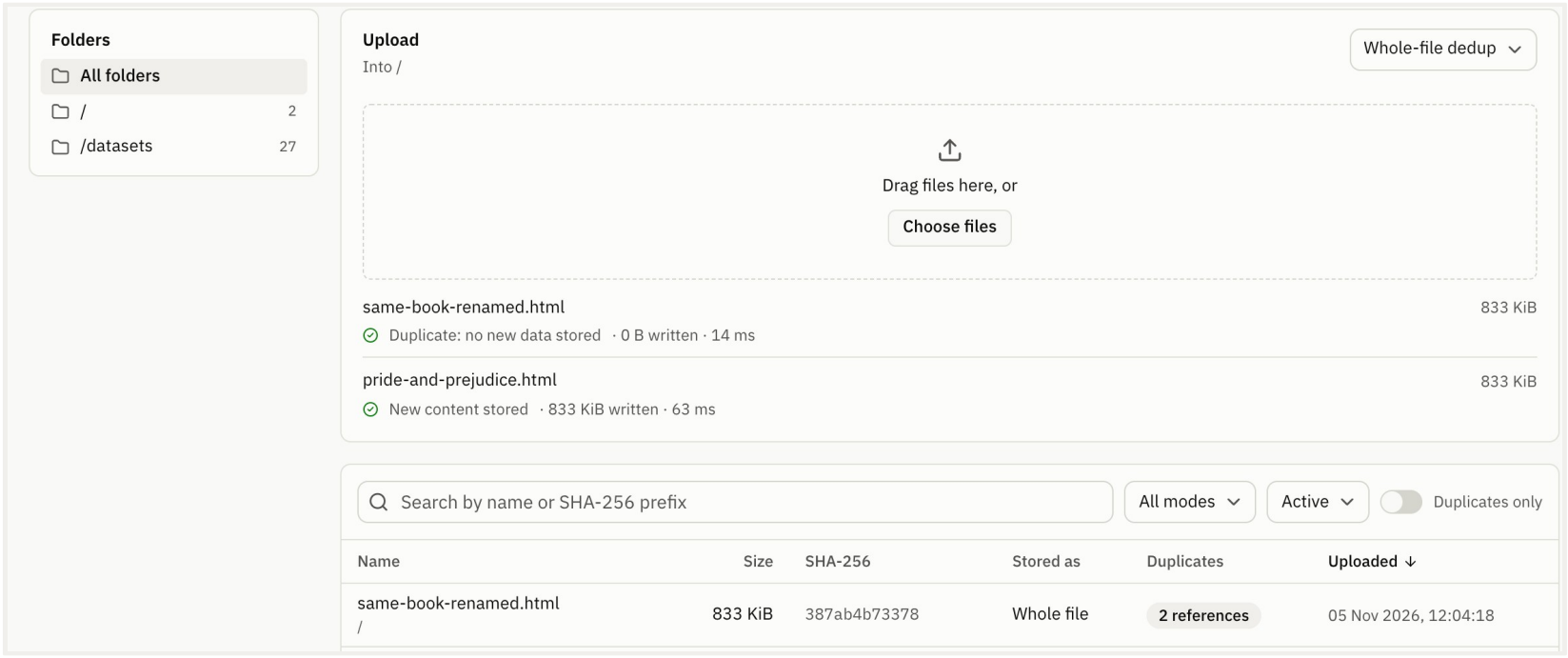
Estimated

A calculation or a prediction, and labelled as one.

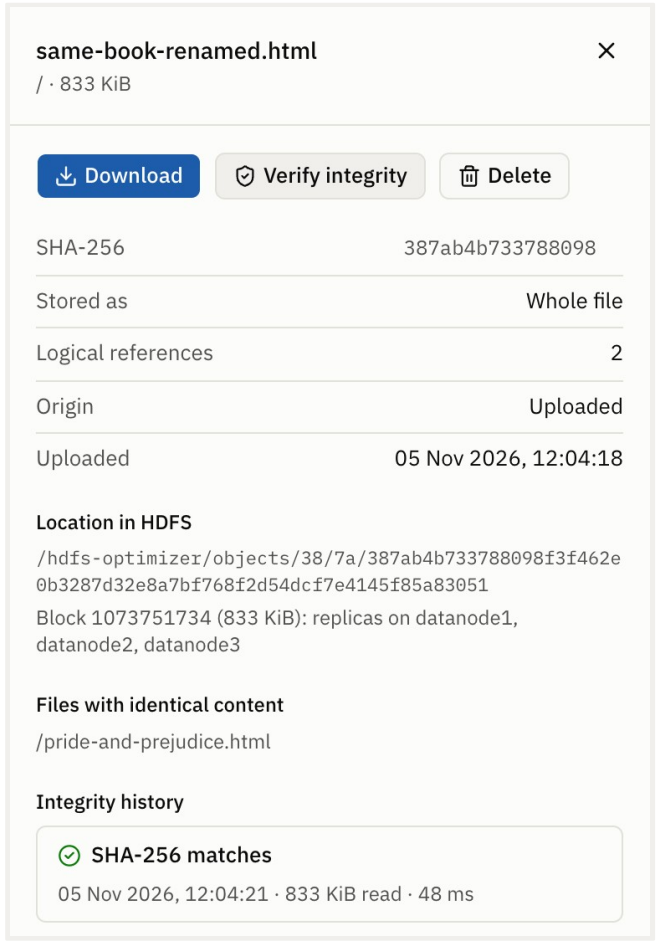
N/A when undefined

An empty system never shows a fake zero.

Upload, browse and inspect files, down to replica locations

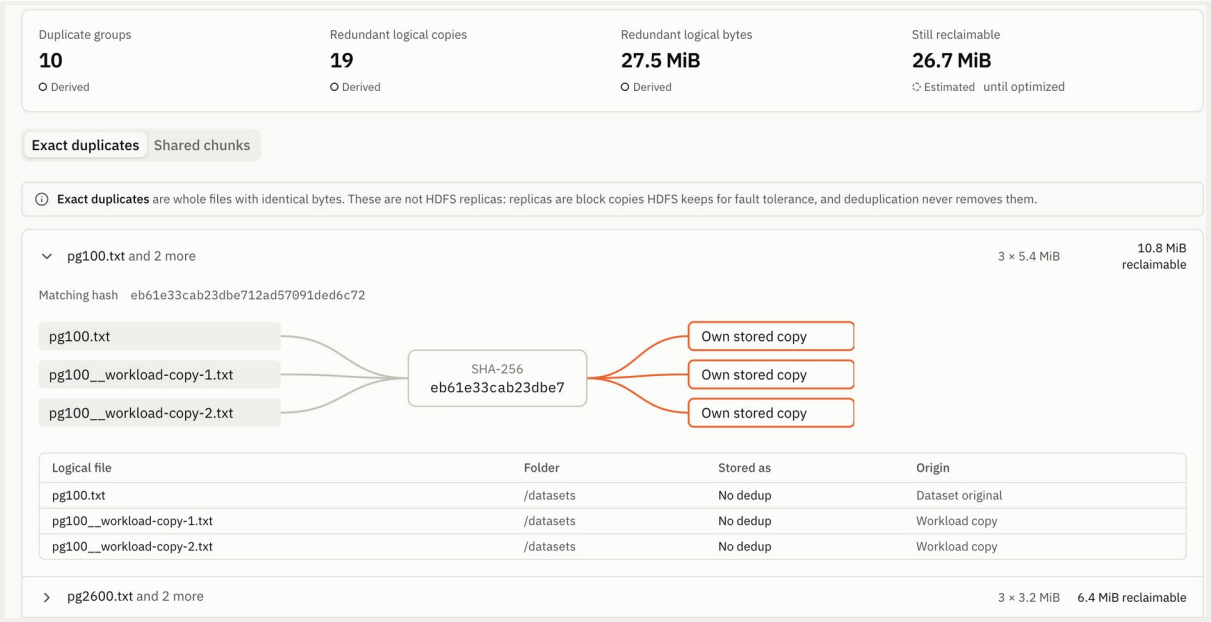


A renamed copy is reported as a duplicate: no new data stored.



Replicas stay on three DataNodes, and the SHA-256 check passes.

Duplicate detection shows how names map to stored bytes



Exact duplicates

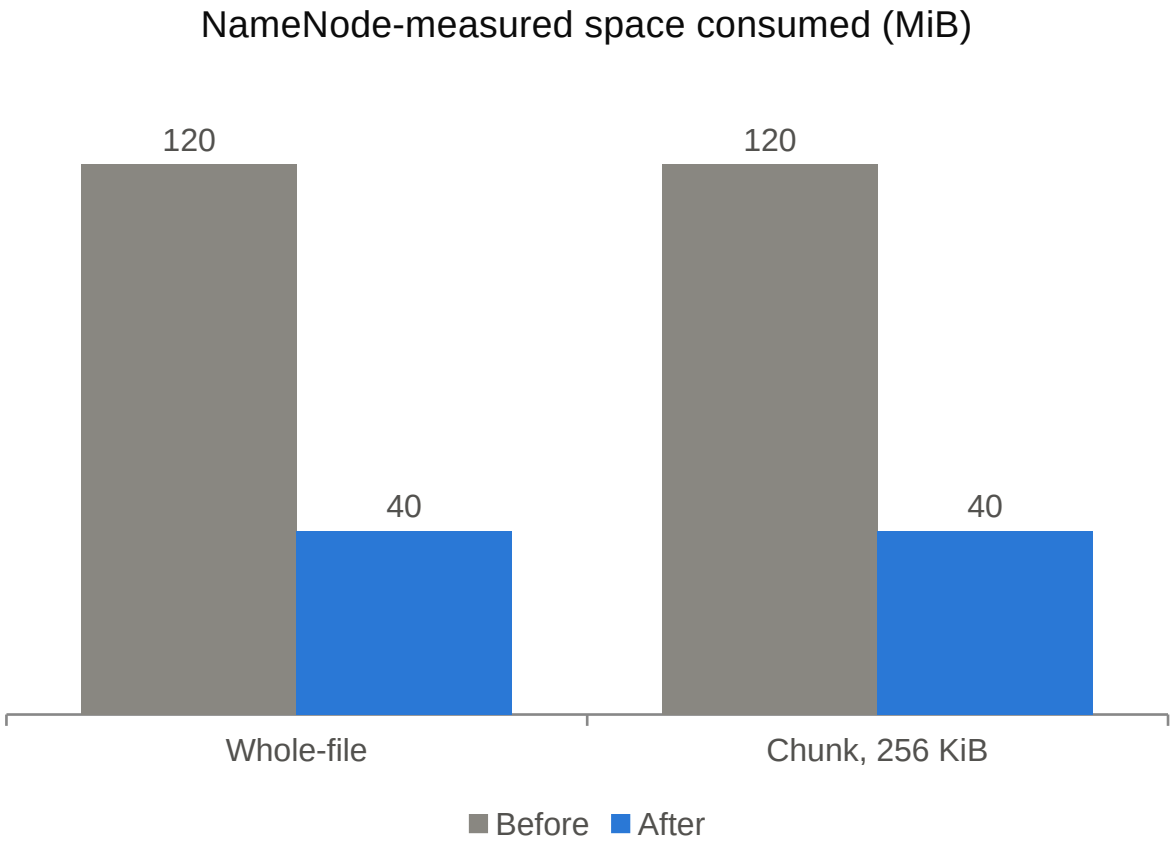
Same SHA-256, different names. In orange, each copy is still stored separately, so its space can be reclaimed.

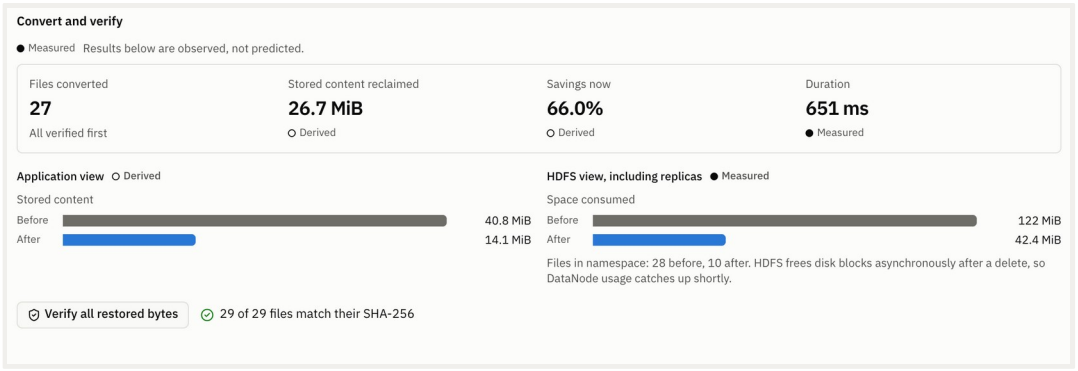


Shared chunks

Blue marks are shared with another file. Truncating or editing one byte shares almost everything. Inserting one byte at the start shares nothing.

Optimization reclaimed the copies and kept every byte





27 of 27

files passed SHA-256 afterwards

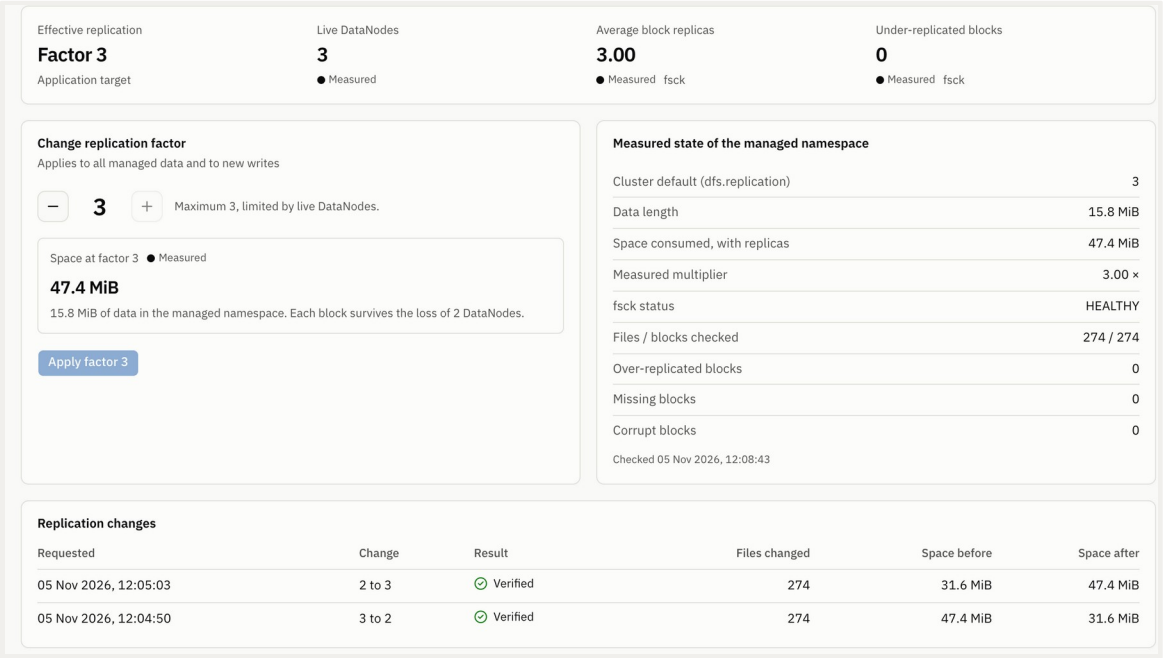
27 to 9

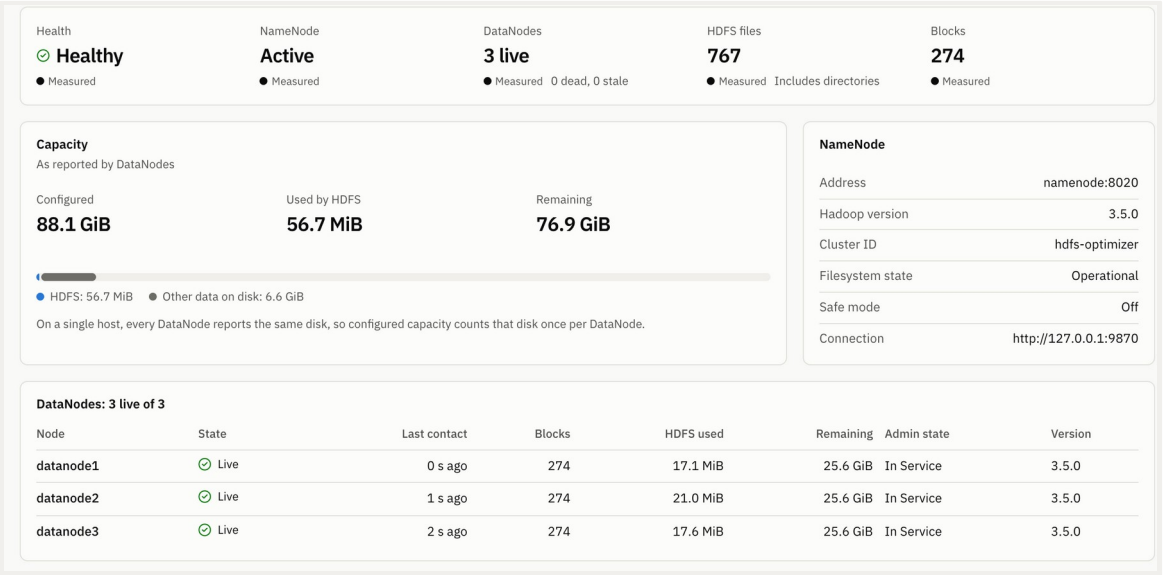
HDFS files, whole-file mode

121 to 40 MiB

on DataNode disks, measured with du

Replication and cluster health come from HDFS itself





Measured at factor 2: space multiplier 2.00, with 0 under-replicated blocks. Success is reported only when fsck and block locations agree. A factor above the live DataNodes is refused.

Authentic files, with provenance for every byte

15

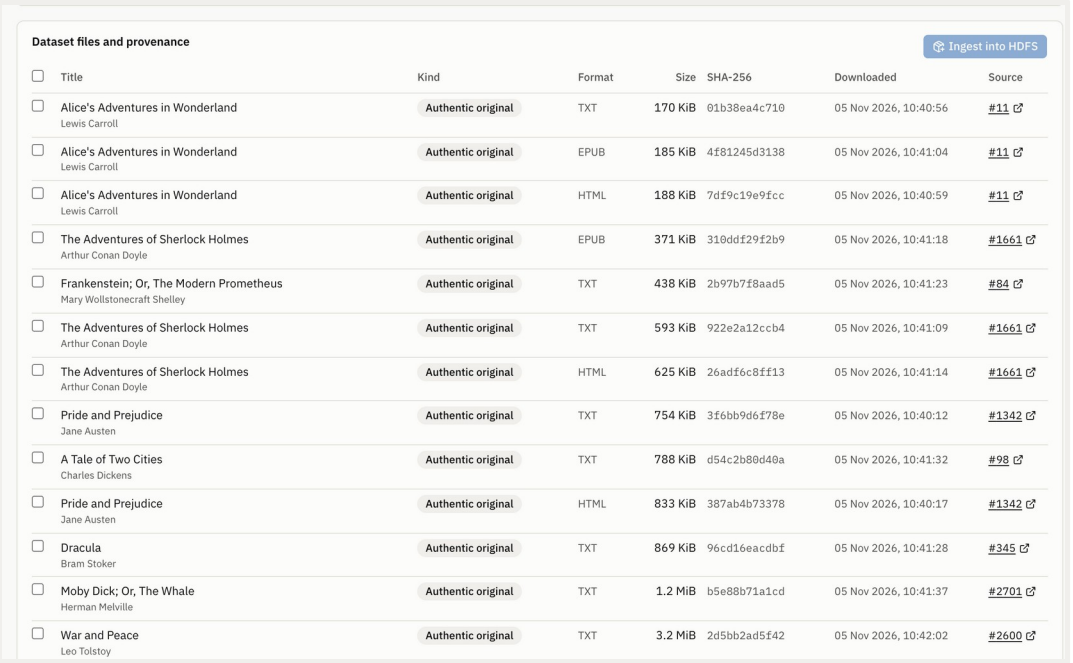
original files from Project Gutenberg

39 MiB

of authentic data, as published

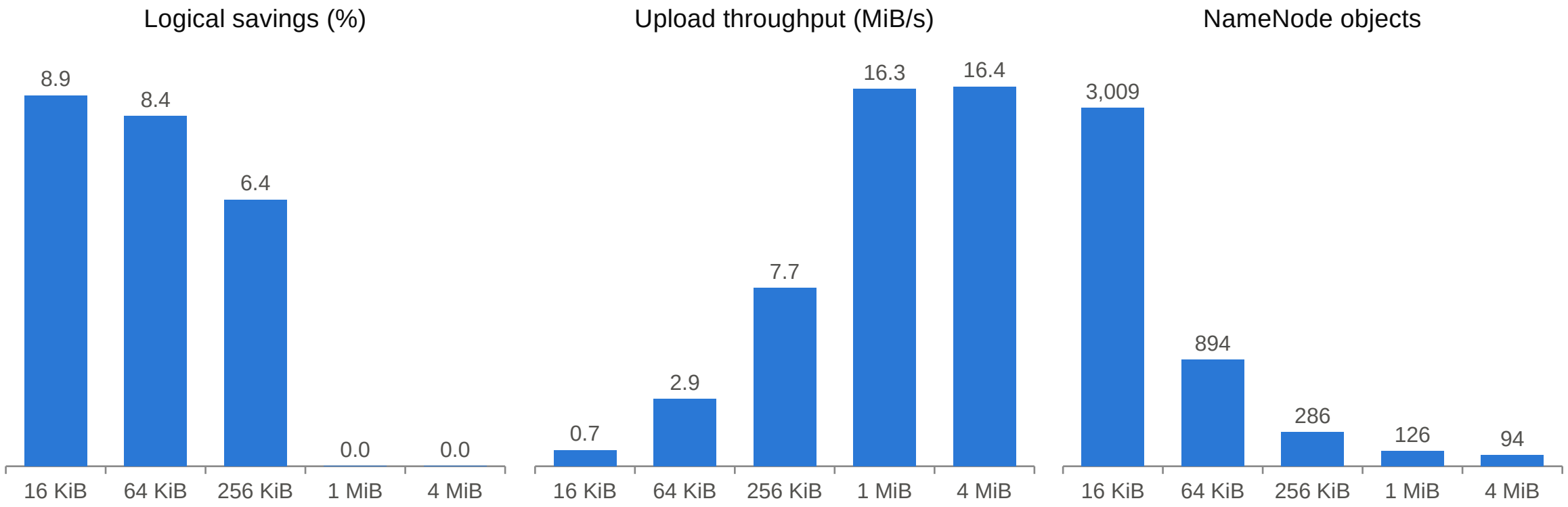
Kind	What it is	Used for
Original	Bytes as published, with URL, date and SHA-256	Everything
Workload copy	Same bytes, another logical name, labelled	Duplicate tests
Transformed	One documented byte edit of one original	Overlap tests

No data was invented. Savings verify the mechanism on chosen workloads; they do not predict real-world savings.



Every file: source page, download URL, date, SHA-256 and kind.

Finer chunks find more overlap, at a steep cost



Logical savings can mislead: at 16 KiB, they read 8.9%, yet disk use measured with du is 14.6% higher than whole-file storage.

Timings: mean of three runs on one laptop. They show the trend, and do not predict a real cluster.

We broke it on purpose, and no data was lost

DataNode stopped

Files stayed readable. A request for factor 3 with two live nodes was refused (HTTP 422).

NameNode stopped

Upload failed with HTTP 503. No file recorded. The retry after restart succeeded (HTTP 201).

Backend killed mid-job

Job marked interrupted. After resume, 27 of 27 files restored byte-identical.

Automated tests that passed

129 unit and API tests

17 on the real HDFS cluster

13 real-browser tests

Corrupt or missing chunks are reported individually, and a corrupt download aborts.

What we learned, what limits it, and what comes next

Engineering challenges

- No transaction spans HDFS and SQLite
- fsck reported success before replicas were deleted
- NameNode JMX values lag by about 10 s
- DataNode deletes are asynchronous, so disk readings need care

Current limits

- Fixed-size chunks: a shift defeats them
- Each chunk is an HDFS file, which costs NameNode memory
- One process and SQLite limit scale
- One laptop: no real hardware fault tolerance

Future work

- Content-defined chunking (FastCDC)
- Packing chunks into container files
- Distributed index; Spark background jobs
- Kerberos, HA and real multi-machine deployment

The largest measured cost, small chunk files, points directly at the next step: packing chunks into containers.

Duplicates gone.
Replicas intact.
Every byte verified.

A working deduplication layer on real HDFS, evaluated with authentic data, including real failures.

Thank you. Questions?

120 MiB

NameNode space consumed before optimization

40 MiB

after, same logical files, same replication

27 of 27

files passed SHA-256 afterwards